Sztuczna Inteligencja – laboratorium nr 2

1. Google Colab.

https://colab.research.google.com/

Należy założyć sobie konto, jeśli nie mamy i zalogować się do systemu.

Colab, lub inaczej „Colaboratory", pozwala pisać i wykonywać kod w języku Python bezpośrednio w przeglądarce dzięki

- brakowi konieczności konfigurowania,
- Bezpłatnemu dostępowi do GPU

Proszę zapoznać się dokładnie z wprowadzeniem w Colaboratory – pierwszy dokument w systemie po otwarciu.

W Colab pracuje się z formatem plików **.ipynb** są to pliki z interaktywnego środowiska Jupyter Notebook wykorzystujące język Python, a dokładniej iPython – czyli interaktywną powłokę dla języka Python.

Poczytać o iPython:

https://ipython.readthedocs.io/en/stable/

Zalecany tutorial:

https://ipython.readthedocs.io/en/stable/interactive/tutorial.html

W Google Colab otwieramy nowy notatnik na dysku. W pierwszej części notatnika (pierwszy blok kodu) importujemy biblioteki na których będziemy pracować:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
print(tf.__version__)
```

```
2.18.0
```

Znak „play” oznacza uruchomienie tego bloku kodu. Bloki kodu można uruchamiać niezależnie unikając niepotrzebnego wielokrotnego wczytywania bibliotek czy uruchamiania kodu, w którym się nic nie zmieniło. Chyba, że wymaga tego logika aplikacji i kolejność wykonywania instrukcji kodu.

Dokumentacja biblioteki Tensorflow:

https://www.tensorflow.org/api_docs/python/tf

Funkcje celu (straty):

https://www.tensorflow.org/api_docs/python/tf/keras/losses

Funkcje optymalizacji:

https://www.tensorflow.org/api_docs/python/tf/keras/optimizers

2. Uproszczona regresja liniowa z użyciem biblioteki Tensorflow.

W tym ćwiczeniu zbudujemy ten sam uproszczony regresyjny model liniowy co na poprzednim laboratorium, jednak tym razem użyjemy biblioteki Tensorflow.

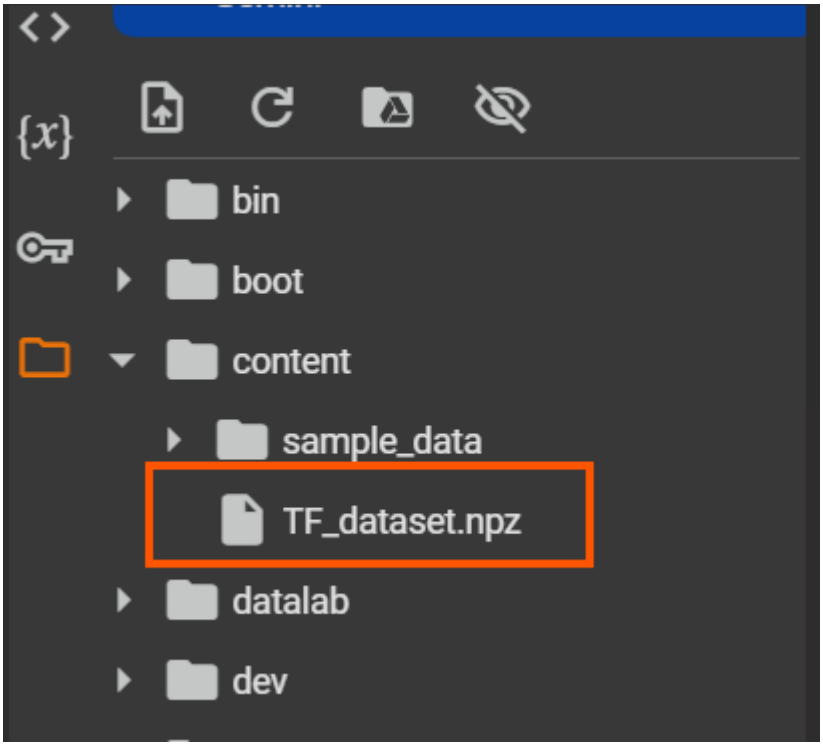
Proszę dokładnie zapoznawać się z załączonymi linkami w ćwiczeniu.

Drugi blok kodu

```
[2] training_data = np.load('TF_dataset.npz')
    input_size = 2
    output_size = 1
```

Pobieramy dane wejściowe i targety dla naszego uproszczonego modelu liniowego, które posłużą do trenowania: **xs, zs, targets**

Plik należy skopiować z poprzedniego projektu i umieścić w analogicznym folderze bieżącego projektu:



Są to próbki, które zostały losowo wygenerowane w poprzednim projekcie i zapisane do pliku w celu ponownego ich użycia:

```
np.savez('TF_dataset', inputs=inputs, targets=targets)
```

Proszę zwrócić uwagę na nazewnictwo.

inputs=inputs oznacza, że w dataset zapisaliśmy do inputs zmienną o takiej samej nazwie. Nazwa pierwszej części jest obligatoryjna, druga jest dowolna. Druga to po prostu nazwa naszej zmiennej, w tym wypadku jest taka sama jak ta, która określa typ danych w dataset.

Analogicznie jest z **targets=targets**

Próbek treningowych mamy także 1000.

Trzeci blok kodu

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(output_size)
])
model.compile(optimizer='sgd', loss='mean_squared_error')
model.fit(training_data['inputs'], training_data['targets'], epochs=100, verbose=1)
```

Teraz tworzymy nasz model.

Model = …… jest to nasza dwuwarstwowa uproszczona sieć neuronowa.

Będziemy korzystać z biblioteki **keras**:

https://www.tensorflow.org/guide/keras?hl=pl

i jej API:

https://www.tensorflow.org/api_docs/python/tf/keras

Jako podstawę naszej sieci użyjemy modelu sekwencyjnego:

https://www.tensorflow.org/guide/keras/sequential_model?hl=pl

Jako, że pracujemy na prostej regresji liniowej – nie używamy w parametrach funkcji aktywacji, która dodaje nieliniowość do sieci neuronowej. Nieliniowość jest oczywiście obligatoryjna przy sieciach wielowarstwowych, ale w tym wypadku jest zbędna, gdyż mamy tylko dwie warstwy: wejście i wyjście. Warstwy wejściowej nawet nie musimy ustawiać – sieć przyjmie nam to co jej podamy. Ustawiamy więc tylko warstwę wyjściową:

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(output_size)
])
```

Jako wyjście ustawiamy najprostszą i standardową jednokierunkową warstwę sieci neuronowej. Ten typ warstwy najczęściej jest używany właśnie jako warstwa wyjściowa.

https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dense

Inne rodzaje warstw:

https://www.tensorflow.org/api_docs/python/tf/keras/layers

By nasza sieć uczyła się prawidłowo potrzebuje dwóch najważniejszych składników: **funkcji straty** do oceniania jakości nauki i **funkcji optymalizacji** do zmieniania parametrów sieci (wagi i bias'y) w trakcie nauki.

```
model.compile(optimizer='sgd', loss='mean_squared_error')
```

Funkcja optymalizacji – użyjemy gotowej z biblioteki, ale tej samej której używaliśmy na poprzednich laboratoriach, czyli **metody gradientu prostego**:

https://www.tensorflow.org/api_docs/python/tf/keras/optimizers/SGD

Funkcja straty także będzie ta sama: **L2-norm**, inaczej strata kwadratowa:

https://www.tensorflow.org/api_docs/python/tf/keras/losses/MeanSquaredError

Nauczmy więc naszą sieć:

```
model.fit(training_data['inputs'], training_data['targets'], epochs=10, verbose=1)
```

- podajemy tu dane treningowe: wejścia i targety
- ustawiamy ilość epok uczenia się: **epochs**
- parametryzujemy sposób wyświetlania procesu uczenia się sieci: **verbose**.

Czwarty blok kodu

Sprawdźmy jak się nasza sieć nauczyła:

```
model.layers[0].get_weights()
```

```
[array([[ 2.0008943],
        [-2.9725385]], dtype=float32),
 array([4.9633536], dtype=float32)]
```

Jak widać dostaliśmy prawidłowe wagi i biasy – te które ustawiliśmy w trakcie generowania danych **targets** w poprzednim ćwiczeniu:

```
targets = 2*xs - 3*zs + 5 + noise
```

Teraz widać, że sieć sama nauczyła się prawidłowych parametrów. W **dataset**, który podaliśmy z pliku tego wzoru już nie było – były tylko wyniki obliczeń – **targets**.

Jeśli byśmy teraz chcieli wykorzystać nauczony model wystarczy podać mu jakieś dane:

```
model.predict_on_batch(training_data['inputs']).round(1)
```

Nie mamy innych niż treningowe więc możemy dla przykładu wrzucić je ponownie.

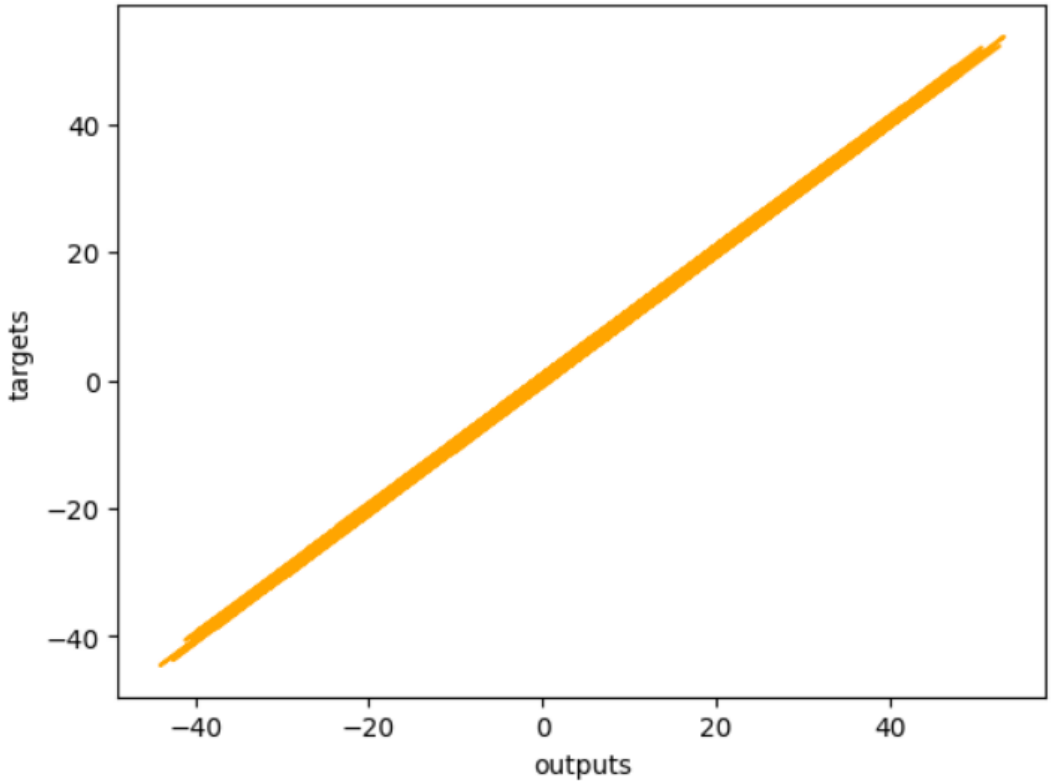
Operujemy tylko na liczbach generowanych losowo więc wyniki nie mają tu żadnego znaczenia oczywiście.

Piąty blok kodu

Dowód na to, że to jest model liniowy 😊. Porównajmy czy nasze wyjścia **y** są zgodne z celami (**targets**), których oczekiwaliśmy w trakcie nauki. Jednak porównajmy dane, które zostaną predykowane na nauczonym modelu, by sprawdzić czy predykcja jest zgodna z naszymi celami:

```
plt.plot(np.squeeze(model.predict_on_batch(training_data['inputs'])),
         np.squeeze(training_data['targets']), color='orange')
plt.xlabel('outputs')
plt.ylabel('targets')
plt.show()
```

Powinniśmy oczywiście otrzymać taki wynik:



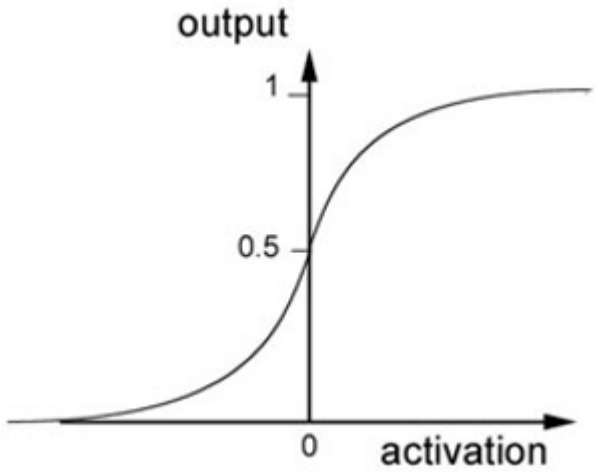
3. Ćwiczenie.

Przeprowadź eksperymenty związane z:

1. Przetestuj predykcje używając innych funkcji optymalizacji i innych funkcji strat – wystarczy skopiować ich nazwy z dokumentacji w odpowiednie miejsce.
2. Spróbuj dodać więcej warstw sieci i ustaw w nich parametr, który zrobi z niej model nieliniowy, czyli funkcję aktywacji, np.:

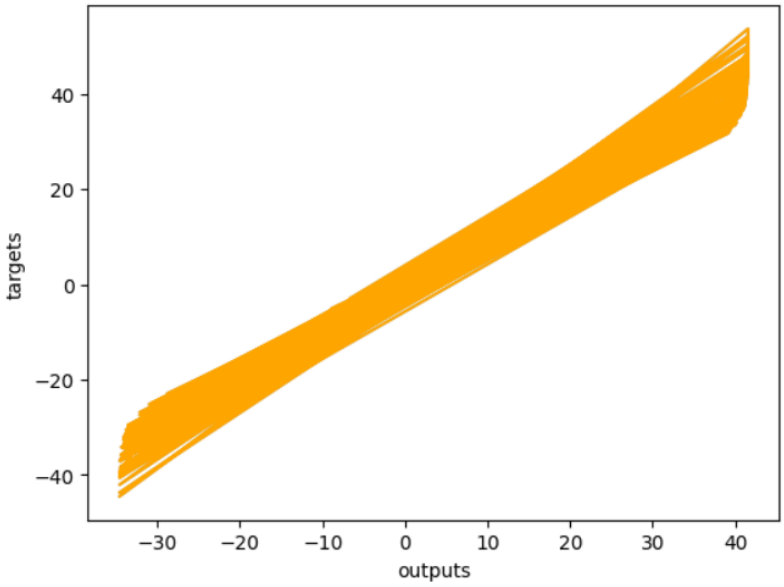
```
tf.keras.layers.Dense(5, activation="sigmoid"),
tf.keras.layers.Dense(5, activation="sigmoid"),
tf.keras.layers.Dense(output_size)
```

w tym przykładzie ilość neuronów na warstwie to **5**, a funkcja aktywacji to **sigmoida:**



która przepuszcza tylko dane większe od 0,5

Nasz model przestał być liniowy:



Parametry wartwy:

```
tf.keras.layers.Dense(
    units,
    activation=None,
    use_bias=True,
    kernel_initializer='glorot_uniform',
    bias_initializer='zeros',
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    lora_rank=None,
    **kwargs
)
```

Warstwa jest domyślnie ustawiona na powyższe parametry. Ich wyjaśnienie:

Args	
`units`	Positive integer, dimensionality of the output space.
`activation`	Activation function to use. If you don't specify anything, no activation is applied (ie. "linear" activation: `a(x) = x`).
`use_bias`	Boolean, whether the layer uses a bias vector.
`kernel_initializer`	Initializer for the `kernel` weights matrix.
`bias_initializer`	Initializer for the bias vector.
`kernel_regularizer`	Regularizer function applied to the `kernel` weights matrix.
`bias_regularizer`	Regularizer function applied to the bias vector.
`activity_regularizer`	Regularizer function applied to the output of the layer (its "activation").
`kernel_constraint`	Constraint function applied to the `kernel` weights matrix.
`bias_constraint`	Constraint function applied to the bias vector.
`lora_rank`	Optional integer. If set, the layer's forward pass will implement LoRA (Low-Rank Adaptation) with the provided rank. LoRA sets the layer's kernel to non-trainable and replaces it with a delta over the original kernel, obtained via multiplying two lower-rank trainable matrices. This can be useful to reduce the computation cost of fine-tuning large dense layers. You can also enable LoRA on an existing `Dense` layer by calling `layer.enable_lora(rank)`.

Przydatna wskazówka: kiedy coś zmieniasz, nie zapomnij o PONOWNYM URUCHOMIENIU wszystkich bloków kodu następujących po zmianie.

Jeśli tego nie zrobisz, algorytm zachowa STARE wartości wszystkich parametrów.